PACKAGEGRAPH

A Recursive Software Dependency & Vulnerability Impact Tracker

DBMS Experiential Learning — Level 3 Project Synopsis

Team Members: Akshata Amara (PES1UG24AM025), Annanya Babu (PES1UG24AM045) | Section: A

Problem Statement

Modern software is built almost entirely on other people's packages — installing one library silently pulls in dozens more as transitive dependencies. When a widely used package discloses a critical vulnerability (e.g. Log4Shell, 2021), teams need to know immediately which of their own packages and services are affected, even through several layers of indirection. A flat 'list of packages' table cannot answer this. PackageGraph models the real dependency graph of a software ecosystem and exposes explosion (what installing a package pulls in) and implosion (what breaks if a package is flagged vulnerable) as recursive database queries, built and validated against real, live package-registry data.

Objectives

- Model packages and dependencies as a directed graph (a package can depend on many, and be depended on by many).
- Implement dependency explosion — recursively resolve a package's full transitive dependency tree.
- Implement vulnerability implosion — recursively trace every package affected by a flagged dependency.
- Generate an impact report atomically whenever a vulnerability is flagged.
- Store heterogeneous registry metadata (README, keywords, maintainers) in a document store, reflecting its genuinely variable shape across ecosystems.
- Deliver a role-based (Security Admin / Developer) Next.js app on a real API layer.

Scope

Covers package/dependency catalog management, transitive dependency resolution, vulnerability flagging, and downstream impact analysis. Excludes automated version-conflict resolution, patch generation, and CI/CD integration.

Dataset & Validated Prototype

Uses live, public data from the npm registry (registry.npmjs.org), which exposes each package's dependencies as JSON at a stable, unauthenticated URL — real data, no scraping or cleanup needed. A working prototype was already validated: crawling 3 real seed packages (express, next, react-dom) to depth 3 pulled 69 real packages and 113 real dependency edges. The explosion query resolved express's full transitive tree — 114 dependencies across 6 real depth levels. The implosion query found that flagging 'debug' as vulnerable would impact 8 real, currently-published packages, including express itself.

Database Design & Justification

Option 1 (SQL + NoSQL): PostgreSQL holds the relational core — Package, Dependency (self-referencing, forming a directed graph), VulnerabilityFlag, ImpactReport, AppUser — chosen for referential integrity, ACID transactions, and native recursive CTE support. MongoDB stores raw package registry metadata (README, keywords, maintainers, license) as documents, since this data genuinely varies in shape across npm/PyPI/Maven — a natural fit for a document store rather than a fixed relational schema.

Advanced Database Features (≥4 required)

- Recursive CTEs/functions — explode_dependencies() and implode_impact() for graph traversal (validated on live data).
- Transactions — atomic vulnerability-flag insert plus impact-report generation.
- Triggers — auto-recompute affected-package count on ImpactReport; block a package from depending on itself.
- Indexing — on Dependency(parent_package_id) and Dependency(child_package_id).

Technology Stack

Frontend: Next.js. Backend/API: Next.js API routes. Databases: PostgreSQL (relational core) + MongoDB (package metadata). Data source: live npm registry API, no scraping or manual dataset cleanup. Version control: Git/GitHub. Auth: role-based (Security Admin, Developer) with bcrypt-hashed passwords and env-based credentials.